



GENERALITAT
VALENCIANA

Conselleria d'Educació,
Universitats i Ocupació

ceedcv

CENTRE ESPECÍFIC
D'EDUCACIÓ A DISTÀNCIA DE
LA COMUNITAT VALENCIANA

Unidad 04 — Sincronización

Bloque 02

2º CFGS DAM · Programación de Servicios y Procesos

Sergi García Barea



Lo que veremos en este bloque



Semaphore

Aforo: hasta N hilos a la vez



Barrier

Fases: nadie empieza hasta que llegan todos



Condition

Avisos: wait() y notify()



Productor-Consumidor

El patrón más usado del mundo real



Deadlock

El abrazo mortal y cómo evitarlo



El ring final

¿Qué mecanismo uso en cada caso?

Semaphore

La idea en una frase

“

Un **Semaphore** permite que hasta **N hilos** accedan al recurso a la vez: es como el **aforo máximo** de un local. Cuando se libera un puesto, entra el siguiente.



El Lock deja pasar a **uno** a la vez. El `Semaphore(2)` deja pasar a **dos**; `Semaphore(3)`, a tres... hasta el número que le pases al constructor.



Un **portero con contador**: `acquire()` lo baja, `release()` lo sube. Mientras queden puestos, se entra.

Semaphore(2) en acción

semaforo.py

Python

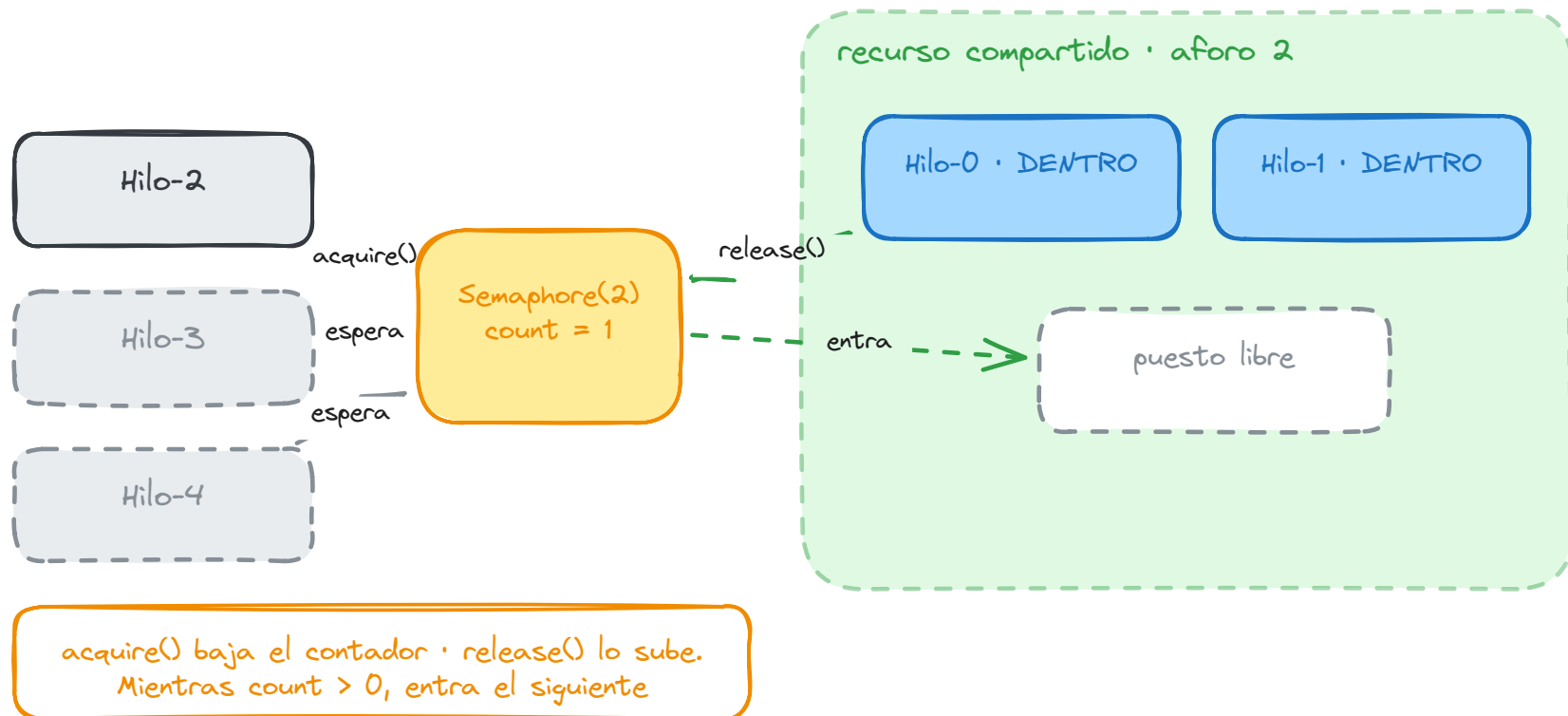
```
import threading, time
semaforo = threading.Semaphore(2) # máx. 2 dentro
def entrar(id):
    print(f"Hilo-{id} esperando...")
    with semaforo:
        print(f" → Hilo-{id} DENTRO")
        time.sleep(2)
        print(f" ← Hilo-{id} SALE")
hilos = [threading.Thread(target=entrar,
                          args=(i,)) for i in range(5)]
for h in hilos: h.start()
for h in hilos: h.join()
```

```
→ Hilo-0 DENTRO
→ Hilo-1 DENTRO
← Hilo-0 SALE
← Hilo-1 SALE
→ Hilo-2 DENTRO
→ Hilo-3 DENTRO
← Hilo-2 SALE
← Hilo-3 SALE
→ Hilo-4 DENTRO
← Hilo-4 SALE
```

Los 5 hilos esperan, pero dentro solo hay **2 a la vez**:
salen de 2 en 2.

El portero con contador

Semaphore(2) — aforo: máximo 2 hilos dentro a la vez



Semáforo con timeout

timeout.py

Python

```
import threading, time
semaforo = threading.Semaphore(1) # solo 1
def entrar(id):
    if semaforo.acquire(timeout=1):
        print(f"Hilo-{id} entró")
        time.sleep(2)
        semaforo.release()
    else:
        print(f"Hilo-{id} TIMEOUT: se va")
hilos = [threading.Thread(target=entrar,
                          args=(i,)) for i in range(3)]
for h in hilos: h.start()
for h in hilos: h.join()
```

```
$ python timeout.py
Hilo-0 entró
Hilo-1 TIMEOUT: se va
Hilo-2 TIMEOUT: se va
```



`acquire(timeout=N)` devuelve **True** si consiguió el recurso y **False** si pasaron los N segundos



A veces no quieres esperar eternamente: con el **False**, el hilo decide **rendirse** en lugar de bloquearse para siempre.

¿Para qué sirve un semáforo?

Situación	¿Por qué un semáforo?
Limitar conexiones a una base de datos	No saturar el servidor de BD
Descargas simultáneas	Máximo N descargas a la vez
Acceso a una API con rate limit	No superar las peticiones permitidas
Impresoras compartidas	Solo las N impresoras disponibles
Aforo de un recurso físico	Como el aforo máximo de un local

  **Diferencia con Lock:** el Lock es `Semaphore(1)` . Para **exclusión mutua** usa Lock (más simple y rápido); para **limitar acceso concurrente** a N recursos, Semaphore.

Mini-chequeo: Semaphore



Responde en 30 segundos:

1. ¿Cuántos hilos dejan entrar `Semaphore(3)` a la vez?
2. ¿Qué devuelve `acquire(timeout=2)` cuando el recurso se libera y cuando no?
3. ¿Cuándo elegirías Semaphore en lugar de Lock?



Respuestas:

- **3 hilos a la vez:** cuando uno sale, el cuarto puede entrar
- **True** si consiguió el recurso dentro de los 2 segundos; **False** si expiró esperando
- Para **limitar acceso concurrente** a N recursos (aforo): conexiones a BD, descargas, rate limit

Barrier

La idea en una frase

“

Una **Barrier** obliga a todos los hilos a esperar hasta que el último llegue. Entonces todos continúan **a la vez**: perfecta para sincronizar **fases** de un trabajo en paralelo.



Lock y Semaphore controlan **cuántos** pasan. La Barrier controla **cuándo**: nadie cruza la meta hasta que el grupo completo está listo.

La carrera de relevos

barrera.py

Python

```
import threading, time
barrera = threading.Barrier(3)
def corredor(id):
    print(f"Corredor-{id} preparándose...")
    time.sleep(id * 0.5) # cada uno tarda distinto
    print(f" → Corredor-{id} en la salida")
    barrera.wait() # espera a los demás
    print(f" 🏁 Corredor-{id} SALIÓ!")
hilos = [threading.Thread(target=corredor,
                          args=(i,)) for i in range(3)]
for h in hilos: h.start()
for h in hilos: h.join()
```

```
→ Corredor-0 en la salida
→ Corredor-1 en la salida
→ Corredor-2 en la salida
🏁 Corredor-0 SALIÓ!
🏁 Corredor-1 SALIÓ!
🏁 Corredor-2 SALIÓ!
```

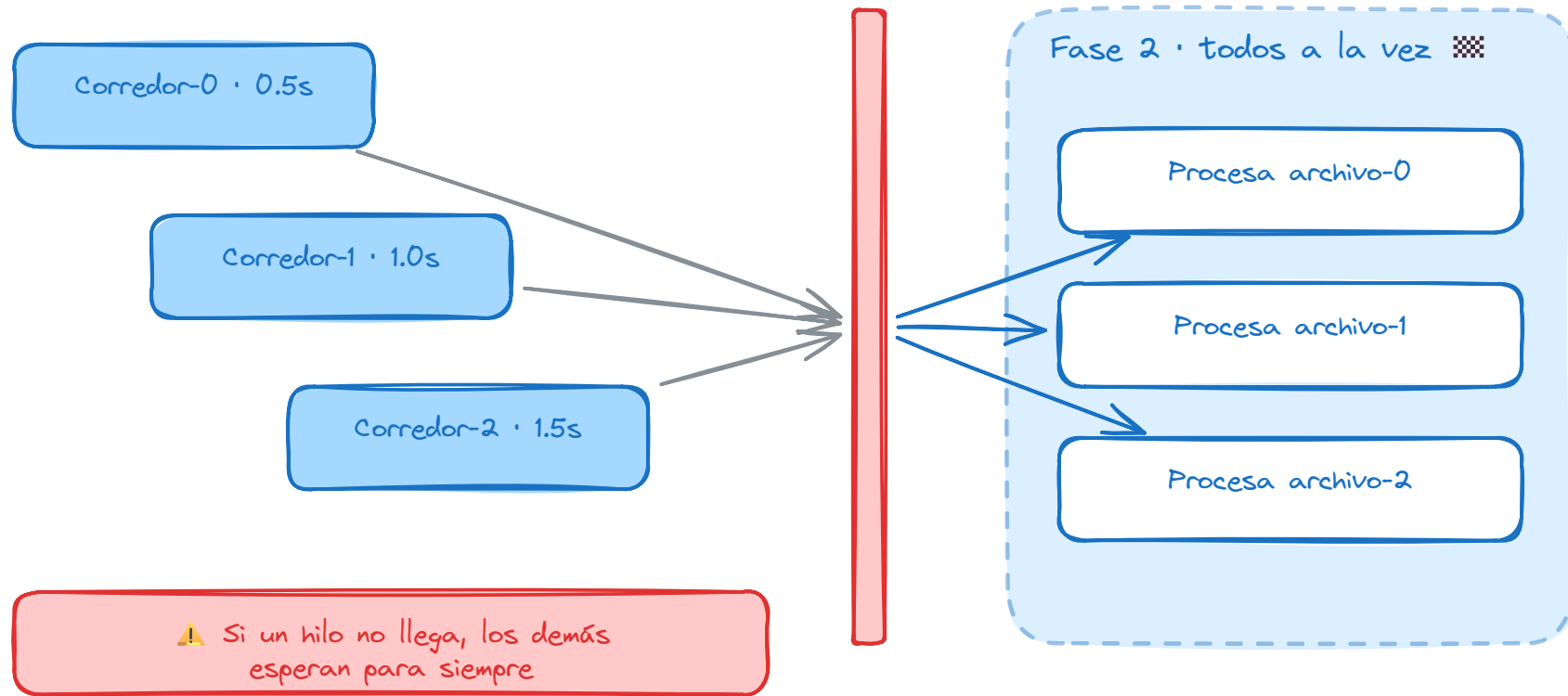
Llegan en momentos distintos, pero **los 3 cruzan a la vez**: `barrera.wait()` devuelve solo cuando el último llega.



Si un hilo no llega, los demás esperan para siempre: **el número de hilos debe coincidir** con el de la barrera.

La barrera de salida

Barrier(3) — nadie cruza hasta que llegan todos
Barrier(3)



Sincronizar fases de un trabajo

fases.py

Python

```
import threading, time
barrera = threading.Barrier(3)
def tarea(id):
    print(f" 📄 Descargando archivo-{id}...")
    time.sleep(1 + id) # duraciones distintas
    barrera.wait() # 🕒 espera a los 3
    print(f" 📄 Procesando archivo-{id} (fase 2)")
hilos = [threading.Thread(target=tarea,
                          args=(i,)) for i in range(3)]
for h in hilos: h.start()
for h in hilos: h.join()
```

📄 Descargando archivo-0...

📄 Descargando archivo-1...

📄 Descargando archivo-2...

📄 Procesando archivo-0 (fase 2)

📄 Procesando archivo-1 (fase 2)

📄 Procesando archivo-2 (fase 2)

✓ Ningún archivo se procesa hasta que **los 3 están descargados**: la barrera convierte el «cada uno a lo suyo» en «todos juntos en cada fase».

Lock vs Semaphore vs Barrier

Mecanismo	Pregunta que responde
Lock	¿Quién toca el recurso? (solo uno)
Semaphore	¿Cuántos a la vez? (hasta N)
Barrier	¿Cuándo empieza la siguiente fase? (cuando todos llegan)

i Los tres se pueden **combinar**: una Barrier para coordinar fases, un Semaphore para limitar el acceso al recurso en cada fase y un Lock para las secciones críticas dentro de cada fase.

Mini-chequeo: Barrier



Responde sin mirar:

1. ¿Qué hace exactamente `barrera.wait()` ?
2. ¿Qué ocurre si solo llegan 2 de los 3 hilos esperados por `Barrier(3)` ?
3. ¿Para qué tipo de problema es la herramienta ideal?



Respuestas:

- Bloquea al hilo hasta que **todos** han llegado a su `wait()` ; entonces todos continúan a la vez
- Los 2 esperan **para siempre**: la barrera nunca se levanta sin el tercero
- Sincronizar **fases**: nadie empieza la fase N+1 hasta que todos terminan la fase N

Condition

La idea en una frase

“

Una **Condition** permite que un hilo se duerma esperando a que otro le **notifique** que algo ocurrió: `wait()` duerme y libera el lock, `notify()` despierta a un hilo dormido.



Lock, Semaphore y Barrier sincronizan **acceso**. La Condition sincroniza **eventos**: espera a que «hay un elemento en la cola» y otro hilo te lo avisa.



Es la pieza que convierte «comprobar sin parar» (quemando CPU) en «**dormir y que me despierten**».

Métodos de Condition

Método	Idea general
<code>wait()</code>	Libera el lock y espera una notificación
<code>notify()</code>	Despierta a un hilo que está esperando
<code>notify_all()</code>	Despierta a todos los hilos que esperan

⚠️ ⚠️ **Siempre dentro de** `with condition:` .
Los tres métodos exigen tener el lock de la condición adquirido.

El Condition lleva asociado un **lock interno**: al entrar en `with condition:` se adquiere...

- 1 `wait()` **libera el lock** mientras duerme, para que otro hilo pueda entrar y producir
- 2 `notify()` despierta al durmiente, que **recomprueba** la condición al salir

i Sin esa liberación al dormir, el productor nunca podría entrar: **deadlock**.

Condition en acción: productor y consumidor

condition.py

Python

```
import threading, time, random
cola = []
condition = threading.Condition()
def productor():
    for i in range(5):
        with condition:
            cola.append(random.randint(1, 100))
            print(f"📦 Producido: {cola[-1]}")
            condition.notify() # avisa al consumidor
def consumidor():
    for _ in range(5):
        with condition:
            while not cola: # ⭐ while, no if
                condition.wait() # libera el lock
            print(f"🍴 Consumido: {cola.pop(0)}")
# h_prod.start() · h_cons.start() · join() de ambos
```

\$ python condition.py

📦 Producido: 73

🍴 Consumido: 73

📦 Producido: 41

🍴 Consumido: 41

...

i El ritmo lo marcan los `sleep()` : a veces el consumidor se adelanta y **espera**; a veces el productor deja varios y el consumidor los come de golpe.

La regla de oro: while, no if

✓ CORRECTO

```
while not cola:  
    condition.wait()
```

Tras despertar, **recomprueba** la condición; si sigue sin cumplirse, **vuelve a dormir**.

✗ INCORRECTO

```
if not cola:  
    condition.wait()
```

Despierta y **asume** que la condición se cumple... pero puede que ya no.

⚠ ¿Por qué? **Falsas activaciones** (spurious wakeups) o que otro hilo haya consumido el elemento mientras esperábamos. 💡 Esta regla separa al novato del profesional: **siempre while después de wait()**.

Otro uso: esperar una señal

señal.py

Python

```
import threading, time
condition = threading.Condition()
listo = False
def trabajador():
    global listo
    with condition:
        while not listo:
            condition.wait() # espera la señal
        print("🔨 Trabajador: ¡a la obra!")
def jefa():
    global listo
    time.sleep(1)
    with condition:
        listo = True
        condition.notify() # ¡despierta!
    print("👤 Jefa: ¡podéis empezar!")
t = threading.Thread(target=trabajador); j = threading.Thread(target=jefa)
t.start(); j.start(); t.join(); j.join()
```

✓ Sin la condición, el trabajador tendría que comprobar `listo` en bucle, gastando CPU. Con `wait()` / `notify()`, duerme hasta que la jefa le avisa.

Situación

¿Por qué Condition?

Cola productor-consumidor

Esperar hasta que haya producto

Recurso que se llena/vacía

Notificar cuando cambia el estado

Turnos entre hilos

`notify()` al que le toca

Mini-chequeo: Condition



Responde en 30 segundos:

1. ¿Qué hace `wait()` exactamente con el lock de la condición?
2. ¿Por qué hay que usar `while` en lugar de `if` después de `wait()` ?
3. ¿Qué diferencia hay entre `notify()` y `notify_all()` ?



Respuestas:

- **Libera el lock** mientras espera (otros hilos pueden entrar) y se queda dormido hasta la notificación
- Por las **falsas activaciones** y porque otro hilo pudo consumir el recurso: con `while` se comprueba y, si no se cumple, se vuelve a dormir
- `notify()` despierta a **uno**; `notify_all()` a **todos**: evita consumidores dormidos para siempre

Productor-Consumidor

La idea en una frase

“

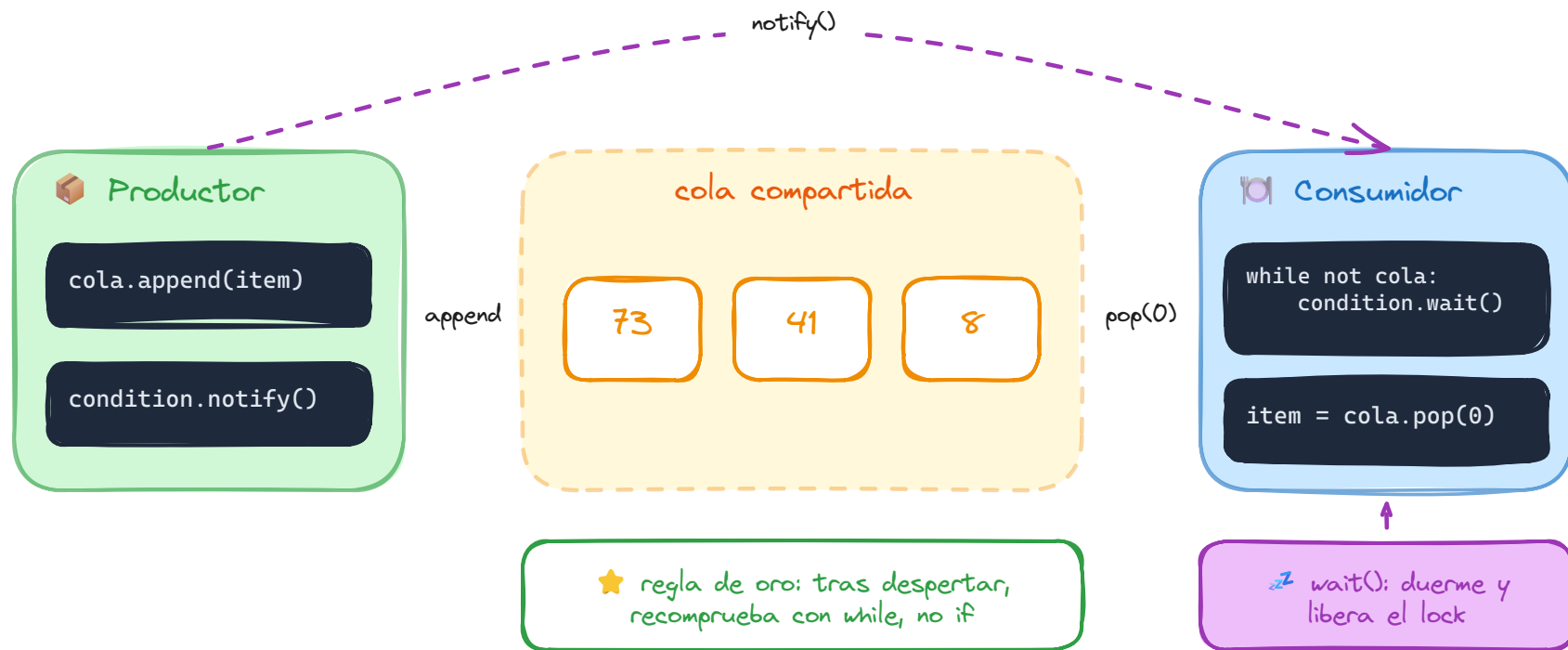
El patrón **productor-consumidor** separa a quien **crea** datos (productor) de quien los **procesa** (consumidor): comparten una cola y una Condition para que el consumidor espere cuando la cola está vacía y el productor le avise cuando añade algo.



Es el patrón de sincronización **más usado en el mundo real**: un hilo llena una cola de tareas y otros hilos las procesan. La Condition es el **pegamento** que los coordina.

La cadena de producción

Productor-Consumidor → cola compartida + Condition



Paso a paso: el baile wait / notify

- 1 El consumidor arranca, la cola está vacía → `with condition:` adquiere el lock → `while not cola:` es True → `wait()` : **libera el lock y se duerme** 
- 2 El productor entra (lock libre) → `cola.append(73)` → `notify()` **despierta al consumidor** 
→ sale del with y libera
- 3 El consumidor despierta → comprueba `while not cola:` → False (¡hay un item!) → `cola.pop(0)` y procesa el 73

⚠ La magia de `wait()` : mientras el consumidor duerme, **libera el lock** para que el productor pueda entrar. Sin esa liberación, nadie produciría y el consumidor esperaría eternamente (**deadlock**).

Con varios consumidores: notify_all()

```
varios_consumidores.py Python

def productor():
    for _ in range(6): # 6 items: 2x3 consumos
        with condition:
            cola.append(random.randint(1, 10))
            print(f"📦 Producido: {cola[-1]}")
            condition.notify_all() # ¡a TODOS!
        time.sleep(1)
# consumidor igual que antes:
# while not cola: condition.wait() → pop(0)
hilos = [threading.Thread(
    target=consumidor, args=(i,))
    for i in range(2)]
hilos.append(threading.Thread(target=productor))
for h in hilos: h.start()
for h in hilos: h.join()
```

⚠ Con `notify()`, si varios consumidores esperan solo se despierta a **uno**: los demás se quedan dormidos **aunque haya trabajo**. Con `notify_all()`, todos recomprueban y el que pueda, consume.

💡 ¿Cola con tope de tamaño? El **semáforo** o la clase `queue.Queue(maxsize=N)` (thread-safe, sin locks a mano) te dan el control: los productores llaman a `put()` y los consumidores a `get()`.

Mini-chequeo: Productor-Consumidor



Responde sin mirar:

1. ¿Qué papel juega cada hilo en el patrón?
2. ¿Por qué `wait()` tiene que liberar el lock mientras el consumidor duerme?
3. ¿Por qué con varios consumidores conviene `notify_all()` ?



Respuestas:

- El **productor** crea datos y los mete en la cola; el **consumidor** los saca y los procesa. Comparten cola y Condition
- Si durmiera **con** el lock puesto, el productor nunca entraría a producir: deadlock. Liberar el lock permite que el ciclo avance
- `notify()` despierta a **uno solo**: los demás pueden quedarse dormidos con trabajo pendiente

Buenas prácticas

La idea en una frase

“

Ya tienes Lock, RLock, Semaphore, Barrier y Condition. Este cierre es el enemigo más temido —el **deadlock**—, las reglas para evitarlo y el «ring» para decidir **qué mecanismo usar en cada caso**.

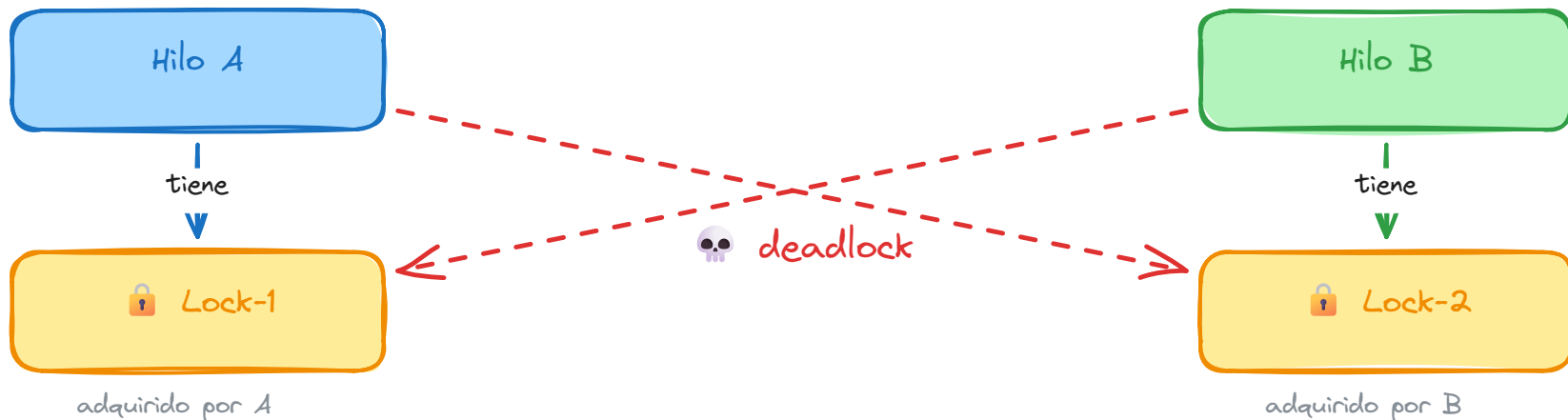


Un **deadlock** es la situación en la que dos o más hilos se esperan **mutuamente para siempre**: cada uno tiene un recurso y espera el que tiene el otro.

El abrazo mortal

Deadlock — el abrazo mortal

⌚ el programa se cuelga para siempre



- ✓ Regla anti-deadlock: todos los hilos adquieren los locks en el mismo orden (primero Lock-1, después Lock-2) y siempre con `with lock: (release())` automático

Deadlock en vivo

deadlock.py

Python

```
import threading, time
lock1 = threading.Lock()
lock2 = threading.Lock()
def hilo_a():
    with lock1:
        time.sleep(0.1) # da tiempo a B
        with lock2:    # B tiene lock2 💀
            print("A: ¡trabajando!")
def hilo_b():
    with lock2:        # ❌ orden invertido
        time.sleep(0.1)
        with lock1:    # A tiene lock1 💀
            print("B: ¡trabajando!")
h1 = threading.Thread(target=hilo_a)
h2 = threading.Thread(target=hilo_b)
h1.start(); h2.start()
h1.join(); h2.join() # ← nunca termina
```

⚠ Los `sleep(0.1)` fuerzan el escenario: A coge `lock1`, B coge `lock2` y después **cada uno intenta coger el del otro**. El programa **se queda colgado** y ni A ni B imprimen su mensaje.

i ¿El arreglo? Que `hilo_b` adquiera `lock1` primero y `lock2` después: **el mismo orden que A**. Nadie puede tener `lock2` sin haber pasado por `lock1`, y el abrazo mortal no ocurre.

Cómo evitar deadlocks: las 3 reglas

- 1 Adquirir los locks siempre en el mismo orden.** Si todos piden primero Lock-1 y luego Lock-2, el abrazo mortal no puede darse
- 2 Usar `with lock:`** (nunca olvidar `release()`). Un release olvidado deja el cerrojo puesto para siempre
- 3 Usar `RLock` si el mismo hilo necesita adquirirlo varias veces:** evita que un hilo se espere a sí mismo

i ¿Y si necesito sincronización **entre procesos** en vez de entre hilos? Usa `multiprocessing.Lock` , `multiprocessing.Semaphore` ...: equivalentes, pero los procesos **no comparten memoria**.

El ring de los mecanismos

Mecanismo	Se define por...	Pregunta que responde
Lock	«Soy el portero: solo dejo pasar a uno cada vez»	¿Quién toca el recurso?
Semaphore(3)	«Soy el guardia de una sala con 3 sill as: cuando se libera una, entra el siguiente»	¿Cuántos a la vez?
Barrier(3)	«Soy el juez de salida: nadie corre hasta que los 3 estén en la línea»	¿Cuándo empieza la fase?
Condition	«Soy el aviso : duermo con wait() y te despierto con notify()»	¿Qué esperas que ocurra?

✓ **Moraleja del ring:** cada mecanismo responde a una pregunta distinta — ¿quién? (Lock), ¿cuántos? (Semaphore), ¿cuándo? (Barrier), ¿qué aviso? (Condition).



Aprieta el lápiz

- 1 **Condición de carrera:** 2 hilos incrementan un contador 1M veces cada uno. Compara con y sin Lock
- 2 **Semáforo descargas:** 10 hilos simulan descargas (sleep 2s) con `Semaphore(3)` . Mide el tiempo total
- 3 **Barrera de fases:** 4 hilos trabajan en 2 fases; la fase 2 no empieza hasta que todos terminan la fase 1
- 4 **Productor-Consumidor:** 1 productor crea números cada 0.5s; 2 consumidores los procesan con Condition
- 5 **Deadlock provocado:** provócalo con 2 hilos y 2 locks... y luego arrégalo con el mismo orden

¿Quién soy? (repaso)



Adivina el mecanismo:

1. Dejo pasar a **un solo** hilo a la sección crítica.
2. El mismo hilo puede adquirirme varias veces sin bloquearse.
3. Guardo una sala con **3 sillitas**: cuando se libera una, entra el siguiente.
4. No dejo correr a nadie hasta que los 3 corredores están en la línea.
5. Duermo con `wait()` hasta que otro me despierta con `notify()`.
6. Soy el bug de dos hilos que se pisan la misma variable.

 **Respuestas:** 1 → **Lock** · 2 → **RLock** · 3 → **Semaphore(3)** · 4 → **Barrier(3)** · 5 → **Condition** · 6 → **condición de carrera**

Resumen del bloque 02



Semaphore(N)

Aforo: contador interno; timeout
opcional



Barrier(N)

Fases: nadie cruza hasta que
llegan todos



Condition

wait() / notify(); siempre
`while` , no `if`



Productor-Consumidor

Cola compartida + notify_all()
con varios



Deadlock

Mismo orden de locks + `with`
`lock:`



El ring

¿quién? · ¿cuántos? · ¿cuándo? ·
¿qué aviso?



Con esto ya sabes **turnarte, limitar, coordinar y avisar**: la caja de herramientas completa de la concurrencia con `threading` .

